

Cryptographie

Chapitre 3

Hachage & MAC

Plan du cours

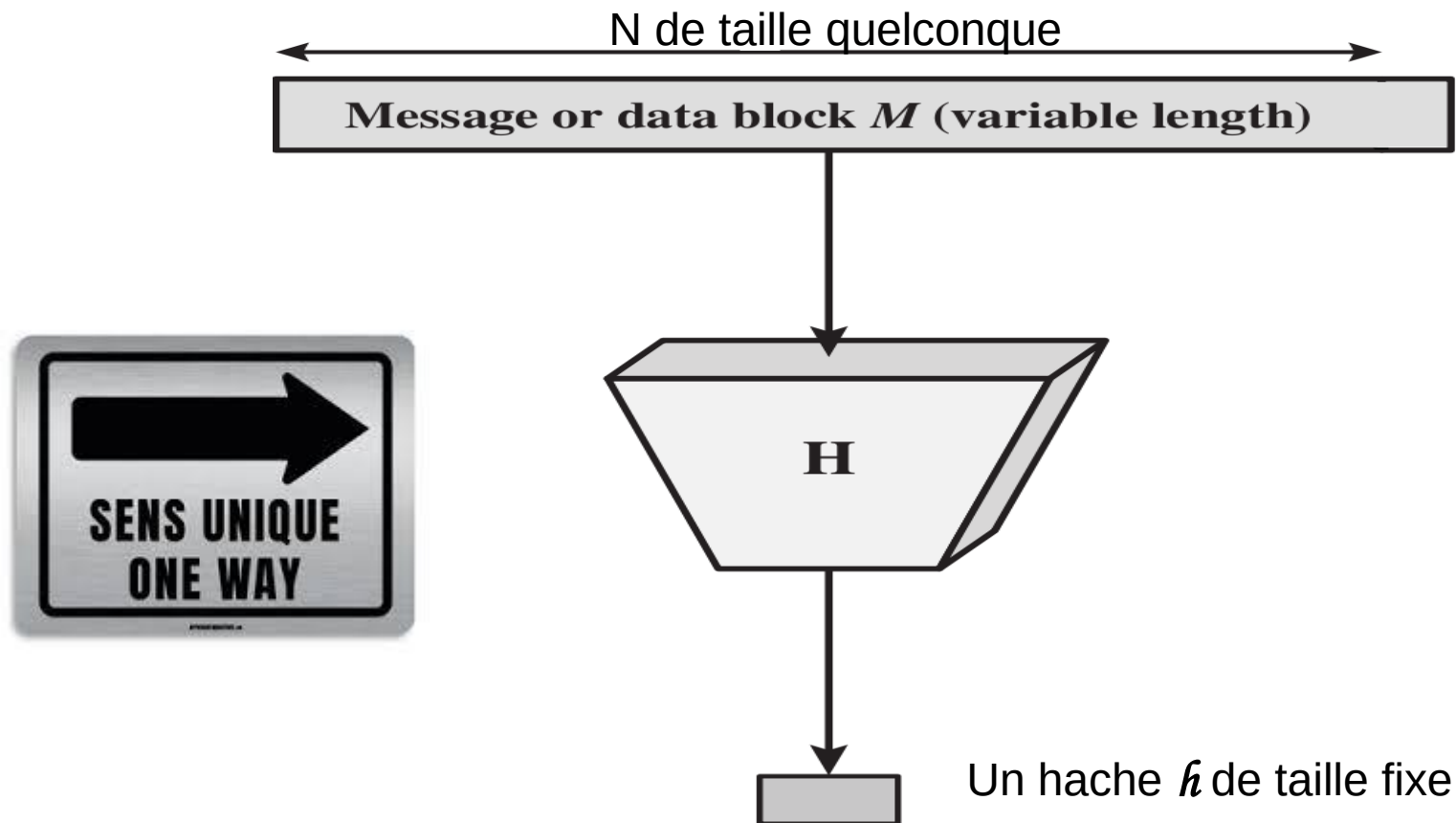
- Introduction à la fonction de hachage
- Propriétés d'une fonction de hachage
- Anniversaire problème
- Application des fonctions d'hachage
- Construction d'un fonction de hachage
- Famille de fonction de hachage MD-SHA

Introduction à la fonction de hachage

- Exemple Motivant:
 - Supposons que Mohamed veut envoyé un message *mes* + la signature de $sig = [mes]_{\text{Mohamed}}$ à Ali.
 - Ali pour vérifier le message $mes = [sig]_{\text{Mohamed}}$
 - Dans le cas d'un message long cela nécessitera un temps de calcul pour que Mohamed signe son message et le joint au message à envoyer. De son coté Ali va dépenser le même temps de calcul pour la vérification de message reçu avec la signature.
 - L'idée sera au lieu de signé le message original en entier on va procéder à la signature de sa compression qui sera de taille réduite que le message ce qui va demander moins de temps de calcul pour Mohamed et Ali.

Introduction à la fonction de hachage

- Une fonction de hachage cryptographique $\mathcal{H}$ est une fonction qui a en entrée un message de taille arbitraire de $\mathcal{N}$ bits et qui produit en sortie une chaîne de taille fixe de n bits et cela indépendamment de la taille du message de départ. $|\mathcal{N}| \gg |n|$.
- Formellement une fonction de hachage est définie comme suit: $\mathcal{H}: \{0, 1\}^* \rightarrow \{0, 1\}^n$



Propriétés d'une fonction de hachage

- On parle de collision entre x et x' lorsque

$$\left\{ \begin{array}{l} x \neq x' \\ H(x) = H(x') \end{array} \right.$$

- Vu que le message peut avoir une taille quelconque du message $> n$, les collisions sont inévitables.
- y est dit empreinte de x tel que $y = H(x)$, et x est appelée antécédent ou préimage de y .

Propriétés d'une fonction de hachage

- Une fonction de hachage cryptographique $h(x)$ doit fournir:
 - La compression : le message résultant de la fonction h est d'une longueur petite.
 - L'efficacité: une empreinte $h(x)$ doit être facile à calculer.
 - Résistance à la préimage (One-way): étant donné y , on ne peut pas trouver en temps raisonnable un x tel que $y = H(x)$.
 - Résistance à la seconde préimage (Weak collision resistance) : étant donné x , on ne peut pas trouver en temps raisonnable $x' \neq x$ tel que $H(x) = H(x')$.
 - Résistance aux collisions (Strong collision resistance) : on ne peut pas trouver en temps raisonnable x et x' tels que $H(x) = H(x')$.

Probability theory, the birthday problem

- Il fait partie des problèmes fondamentaux de la cryptographie (dans les attaques par force brute).
- Soit l'énoncé suivant :
- ❖ Supposons que je suis dans un amphi entrain de donner un cours avec un nombre d'étudiant N .

Question 1 : que devra être le nombre d'étudiant N pour qu'il ait une chance de trouver un étudiant avec la même date d'anniversaire que Moi avec une probabilité supérieur à $1/2$?

Probability theory, the birthday problem

- Réponse1: on peut répondre à cette question en posant la question inverse qu'elle est la probabilité pour qu'il n'ait aucun étudiant avec la même date de naissance que moi:
 - La probabilité pour qu'un étudiant n'ait pas la même date de naissance que moi $Pr1 = 364/365$.
 - La probabilité pour qu'aucun des étudiants n'aient la même date d'anniversaire et le produit de N fois de $Pr1$.
 - $Pr2 = 364/365 * 364/365 * 364/365 \dots \dots \dots * 364/365 = (364/365)^N$
N fois
 - La probabilité pour qu'il ait une personne avec la même date d'anniversaire que moi $Pr3 = 1 - Pr2 = 1 - (364/365)^N$
 - Pour trouver N il suffit de remplacer $Pr3$ par $1/2$ ça donne :
 $1/2 = 1 - (364/365)^N \Rightarrow 1/2 + (364/365)^N = 1 \Rightarrow (364/365)^N = 1/2$.
- Pour trouver la solution il suffit de faire entrer log ça nous donne:
- $$N \log(364/365) = \log(1/2) \Rightarrow N \approx -0,301 / -0,00119 \approx 252,94 \approx 253$$
- Résistance à la seconde préimage .

Probability theory, the birthday problem

- Il fait partie des problèmes fondamentaux de la cryptographie (dans les attaques par force brute).
- Soit l'énoncé suivant :
- ❖ Supposons que je suis dans un amphi entrain de donner un cours avec un nombre d'étudiant N .

Question 2: que devra être le nombre de personne N pour qu'il ait deux personnes ou plus qui partage la même date d'anniversaire dans l'amphi avec une probabilité supérieur à $1/2$?

Probability theory, the birthday problem

- Réponse1: Nous allons raisonner de la même manière avec le complément qu'elle est la probabilité pour les personnes se trouvant dans l'amphi ne partage pas la même date d'anniversaire :
- La probabilité pour que sur les N personne n'aient pas la même date de naissance

$$\text{Pr1} = 365/365 * 364/365 * 363/365 * 362/365 * 361/365 * \dots * (365-N+1)/365.$$

$$\text{Pr1} = (365! / (365-N)!) / 365^N.$$

- La probabilité pour qu'il ait des personnes avec la même date d'anniversaire est $\text{Pr2} = 1 - \text{Pr1} = 1 - (365! / (365-N)!) / 365^N$.
- Pour trouver N il suffit de remplacer Pr3 par 1/2 ça donne :

$$1/2 = 1 - (365! / (365-N)!) / 365^N \Rightarrow 1/2 + (365! / (365-N)!) / 365^N = 1 \Rightarrow$$

$$(365! / (365-N)!) / 365^N = 1/2. \text{ On peut aussi écrire ça comme suit : } 1/2 = \frac{\prod_{i=0}^{N-1} 365-i}{365^N}$$

N=23. Résistance aux collisions

Application des fonctions de hachage

- Authentification (HMAC)
- Intégrité des messages (HMAC)
- Empreinte digitale du message
- Détection de la corruption des données
- Efficacité de la signature numérique
- Stockage des mot de passe (one way password file)
- Et encore d'autres fonctions

Construction d'un fonction de hachage

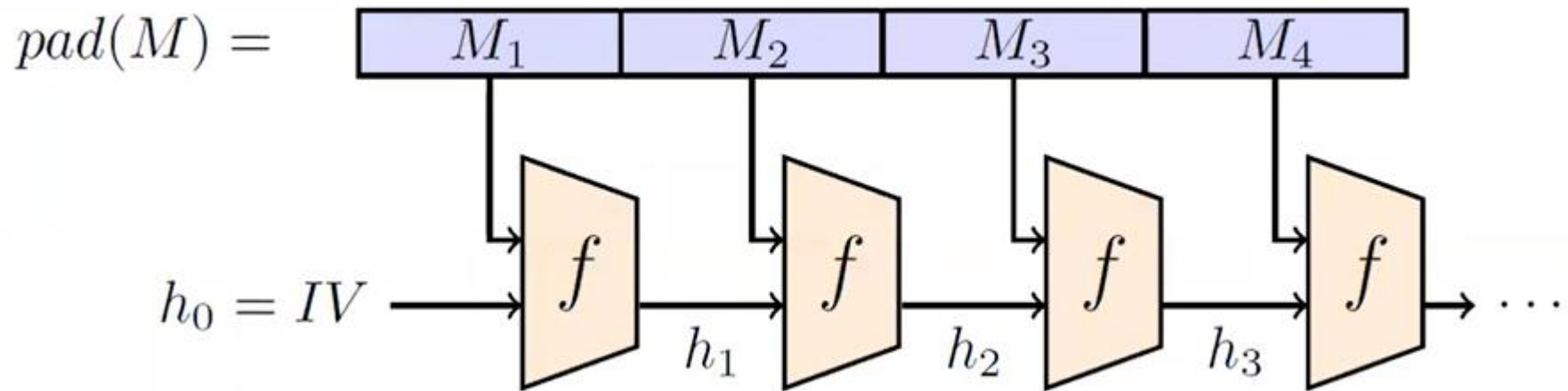
- De nombreuses fonctions de hachage adoptent une conception itérative pour s'adapter à une entrée de longueur variable.
- Le plus connu de ce type de constructeur itérative et le constructeur de **Merkle-Damgard**:
 - Le message M est divisé en un nombre de bloc fixe $\mathbf{M}=(\mathbf{m}_0, \mathbf{m}_1, \dots, \mathbf{m}_{n-1})$. Il est préférable d'utiliser un padding (expansion) sur le message afin d'avoir un nombre de bits correspondant à la taille d'un bloc.
 - Chaque bloc du message est compressé à l'aide d'une fonction de compression l'un après l'autre dont résulte $\mathbf{h}_i$ qui est appelé en anglais chaining variable qui sera utilisé dans l'étape de compression $\mathbf{i}+1$ du bloc $\mathbf{m}_{i+1}$.
 - $\mathbf{h}_0$ est fixe et il fait partie des caractéristique de la fonction de hachage.

Construction d'un fonction de hachage

- De nombreuses fonctions de hachage adoptent une conception itérative pour s'adapter à une entrée de longueur variable.
- Le plus connu de ce type de constructeur itérative et le constructeur de **Merkle-Damgard**:
 - Le message M est divisé en un nombre de bloc fixe $\mathbf{M}=(\mathbf{m}_0, \mathbf{m}_1, \dots, \mathbf{m}_{n-1})$. Il est préférable d'utiliser un padding (expansion) sur le message afin d'avoir un nombre de bits correspondant à la taille d'un bloc.
 - Chaque bloc du message est compressé à l'aide d'une fonction de compression l'un après l'autre dont résulte $\mathbf{h}_i$ qui est appelé en anglais chaining variable qui sera utilisé dans l'étape de compression $\mathbf{i}+1$ du bloc $\mathbf{m}_{i+1}$.
 - $\mathbf{h}_0$ est fixe et il fait partie des caractéristique de la fonction de hachage.

Construction d'un fonction de hachage

- $f: \{0, 1\}^b \times \{0, 1\}^n \rightarrow \{0, 1\}^n$. b est la taille du bloc et n la taille de Haché intermédiaire (chaîning variable)
- La valeur initiale de IV est d'une taille n bits



- Si $f(m_i)$ est résistante au collision alors $F(M)$ l'est aussi

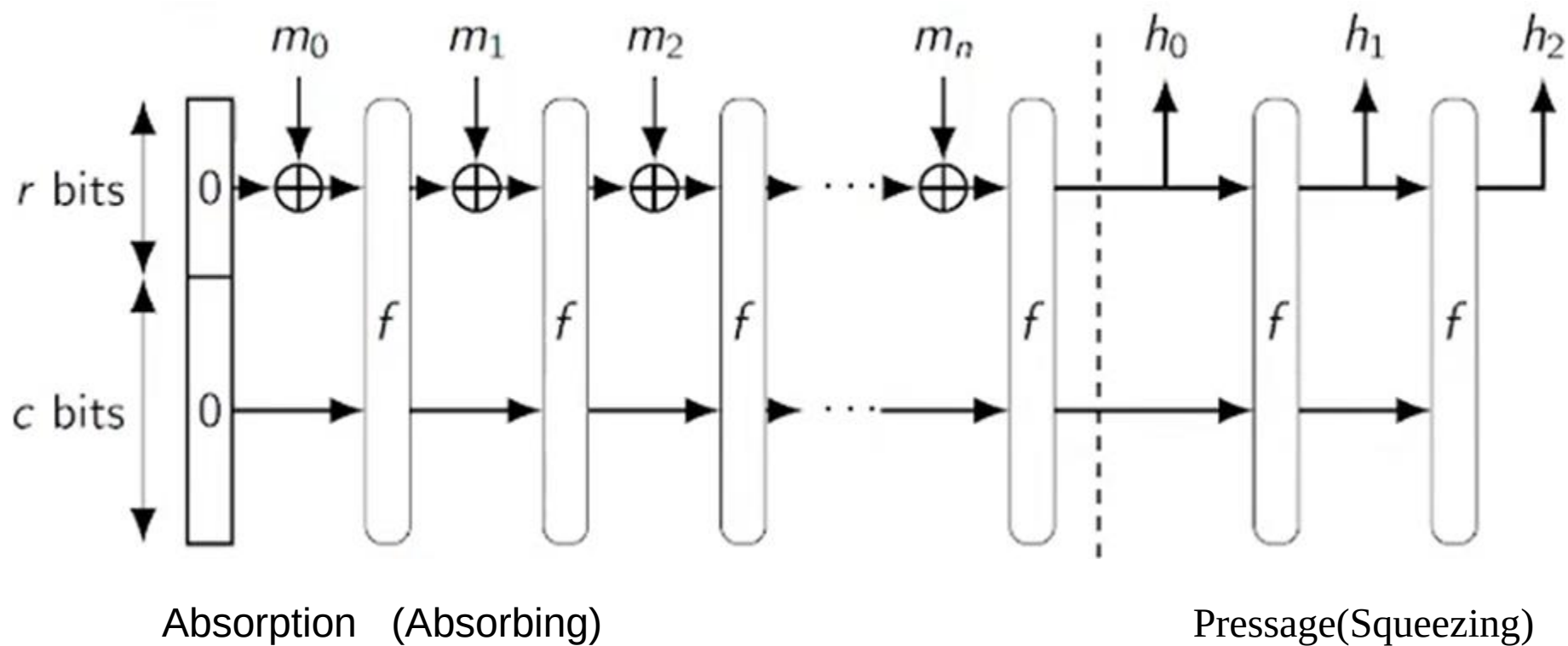
Construction d'un fonction de hachage

Une autre construction de fonction de hachage a été proposée et utilisée par les auteurs le vainqueur du concours SHA3

G. Bertoni, J. Daemen, M. Peeters, and G. Van Assche, Cryptographic sponge functions, January 2011, <http://sponge.noekeon.org/CSF-0.1.pdf>.

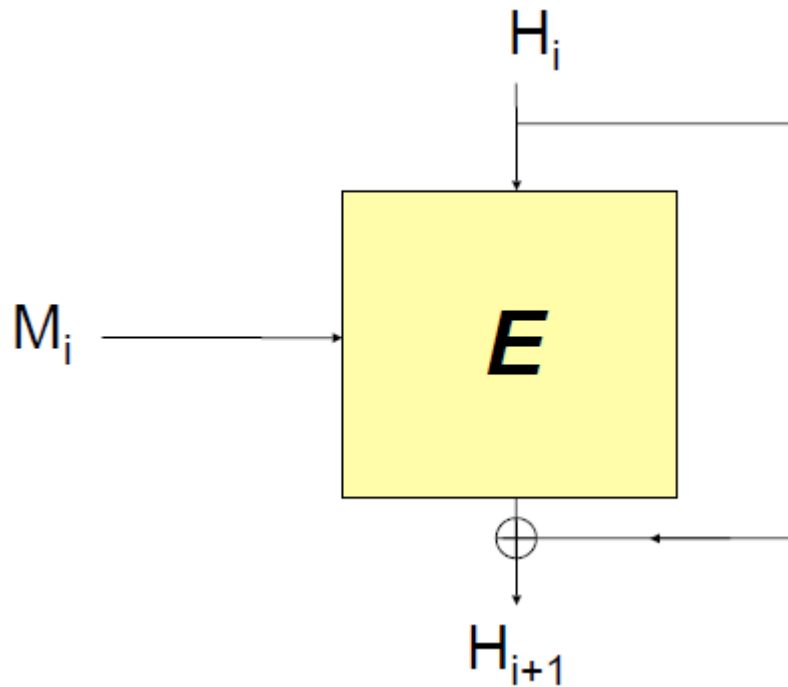
- Un état interne de $r + c$ bits est utilisé.
- On note $m_0, \dots, m_{\ell-1}$ le message avec padding, découpé en blocs de r bits.
- À chacun des ℓ tours, le bloc m_i est ajouté bit à bit aux r premiers bit de l'état, une fonction de permutation non-linéaire et non-inversible, appelée fonction de compression. Cette opération est répétée jusqu'à ce que toutes les données d'entrée aient été absorbées.
- Une fois que tout le message a été traité, le haché est produit sur plusieurs tours : à chaque tour les r premiers bits de l'état donnent une partie du haché ($h_0, h_1, \dots$) et la fonction f est appliquée sur l'état (c'est la partie où l'on presse l'éponge).

Construction d'un fonction de hachage



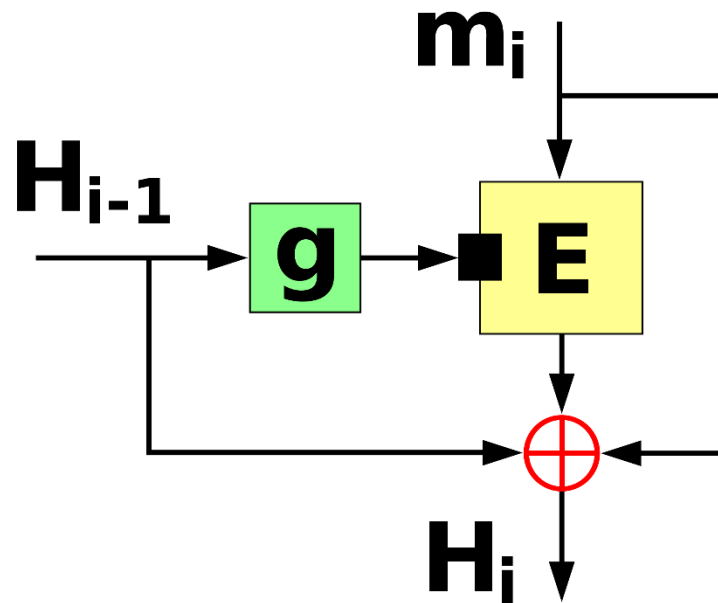
Construction d'un fonction de hachage

- Pour construire une fonction de compression, on peut utiliser la construction de Davies et Meyer qui utilise un chiffrement par bloc : on a $f(m_i || h_i) = \text{Encrypt}_{m_i}(h_i) \oplus h_i$ (le rajout de h_i permet de complexifier les collisions)



Construction d'un fonction de hachage

- Un autre constructeur à base de chiffrement à bloc La construction de Miyaguchi-Preneel est une amélioration de la construction précédente, particulièrement robuste d'un point de vue cryptographique.
- La fonction g est une fonction d'adaptation à la taille de clef de la fonction de chiffrement E . On a ainsi $\mathbf{h}_i = \mathbf{E}_{g(\mathbf{h}_{i-1})}(\mathbf{M}_i) \oplus \mathbf{h}_{i-1} \oplus \mathbf{M}_i$.

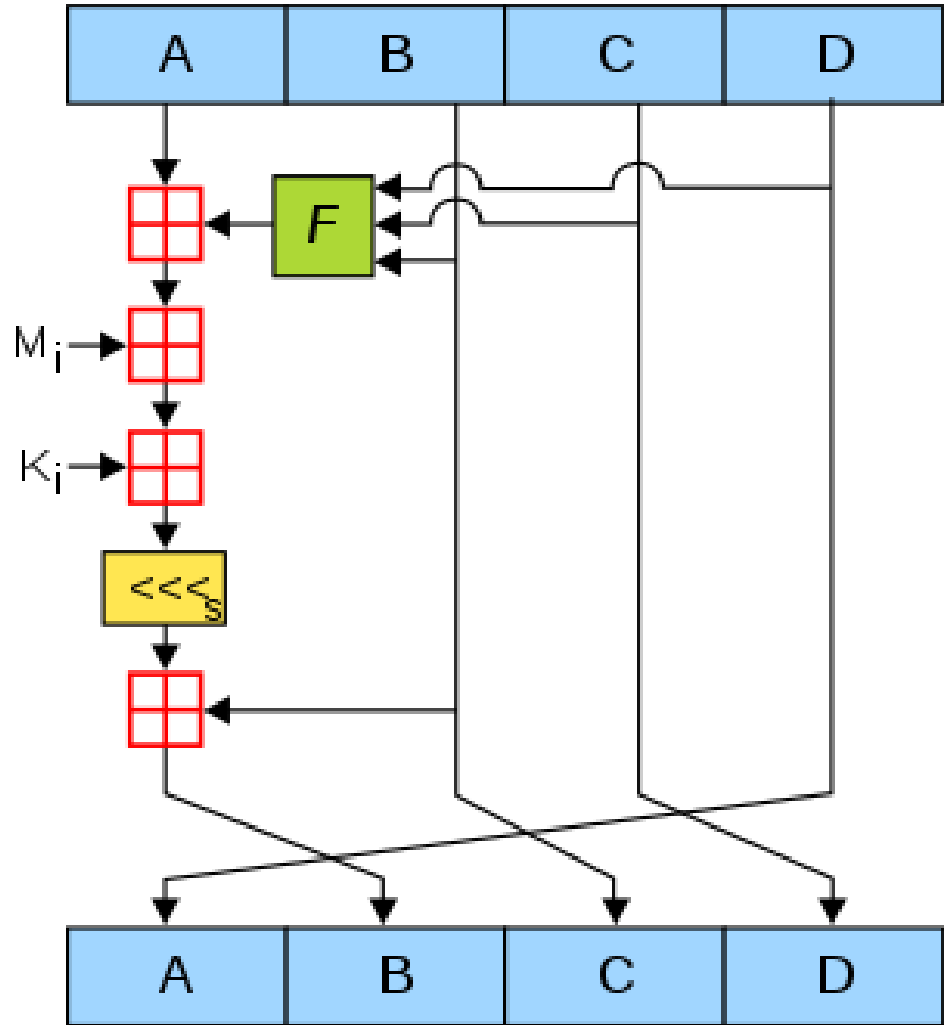


Famille de fonction de hachage MD-SHA

- Les fonctions de hachage les plus connues et utilisées appartiennent à la famille MD-SHA qui regroupe entre autres les fonctions MD4, MD5, HAVAL, RIPE-MD, SHA-0 et SHA-1.

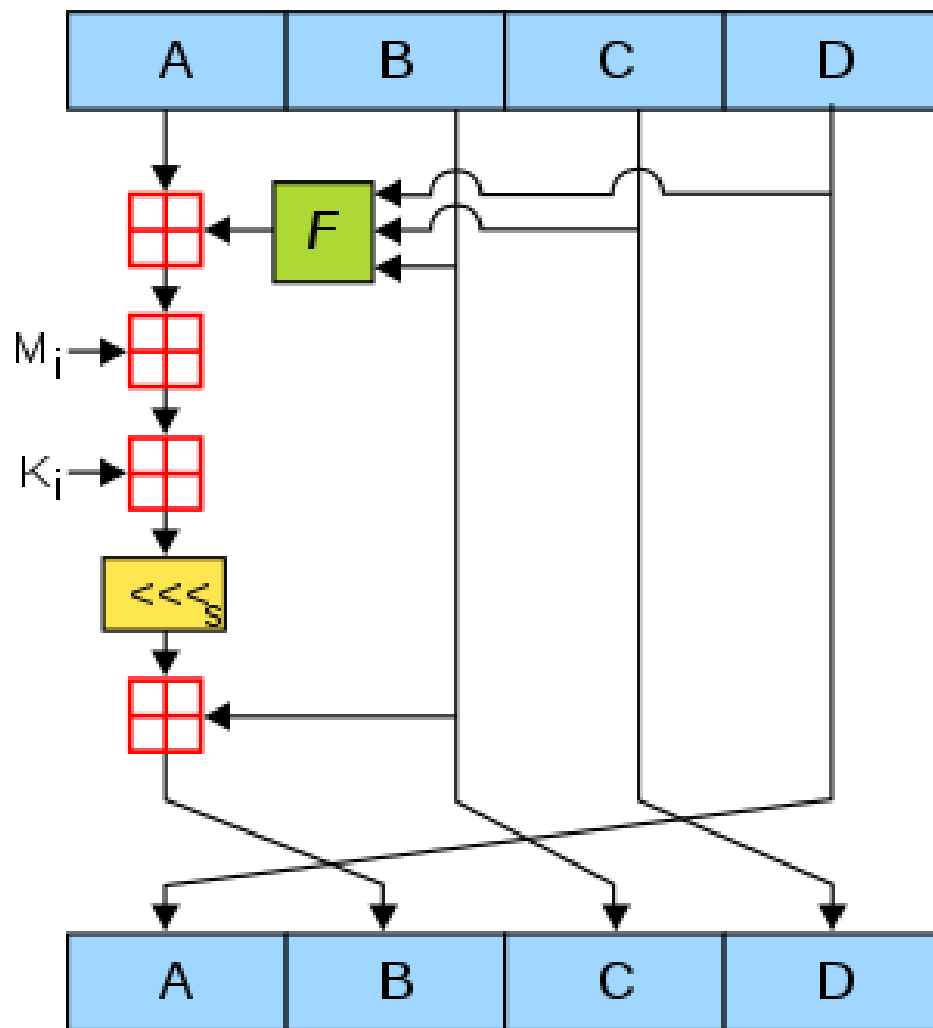
Famille de fonction de hachage MD-SHA

- MD5 : Message Digest 5 développé par Ronald Rivet en 1991 afin de remplacer MD4
 - Taille du bloc : 512 bits.
 - Taille de l'emprunte : 128 bits
 - Chaque bloc est divisé en 16 sous bloc de 32 bits chacun.
 - Type : Merkle-Damgard.
 - Nombre de round : 64.



Famille de fonction de hachage MD-SHA

- Etat interne 4 blocs A;B; C;D de 32 bits.
- M_j l'un des 16 petits blocs 32 bits, varient aussi : $M_1; M_1; M_{16}$.
- K_i constante de 32 bits, il y en a 64.
- Rotation de s bits vers la gauche, s constante entre 1 et 64, il y en a 64.
- On a $4 \times 16 = 64$ tours au total pour une

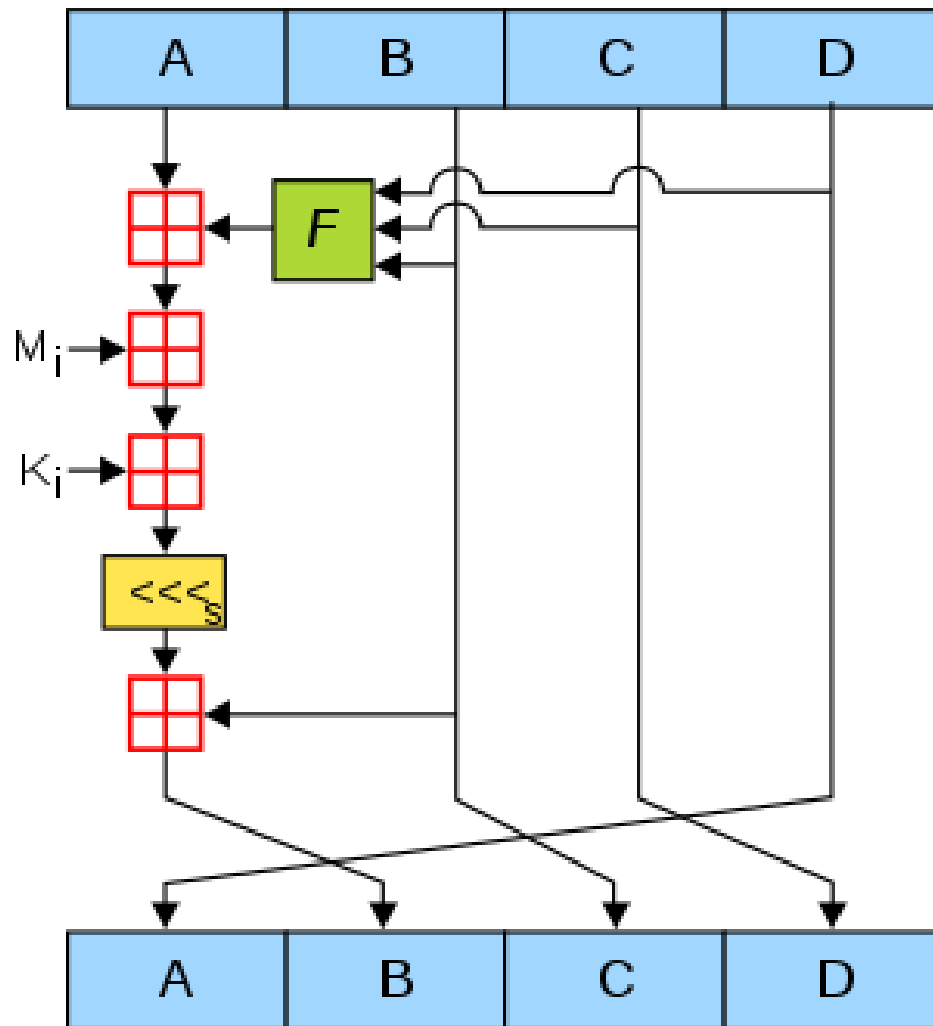


Famille de fonction de hachage MD-SHA

- MD5 : Message Digest 5.

➤ Les tours sont divisés en 4 paquets de 16 sous-tours sur lesquels sont appliquées 4 fonction :

1. $F = (B \text{ AND } C) \text{ OR } (\overline{B} \text{ AND } D)$
2. $G = (D \text{ AND } B) \text{ OR } (\overline{D} \text{ AND } C)$
3. $H = B \oplus C \oplus D$
4. $I = C \oplus (B \text{ OR } \overline{D})$



Famille de fonction de hachage MD-SHA

- Sécurité de MD5. La taille d'empreinte dans MD5 est de 128 bits. La recherche de collisions par force brute (attaque de Yuval) a donc une complexité de 2^{64} calculs d'empreintes. Pourtant, plusieurs algorithmes ont été proposés récemment pour trouver des collisions avec MD5 en seulement 2^{42} opérations, voire 2^{30} . C'est pourquoi MD5 n'est plus considéré comme sûr aujourd'hui.

MD5 Collision Example

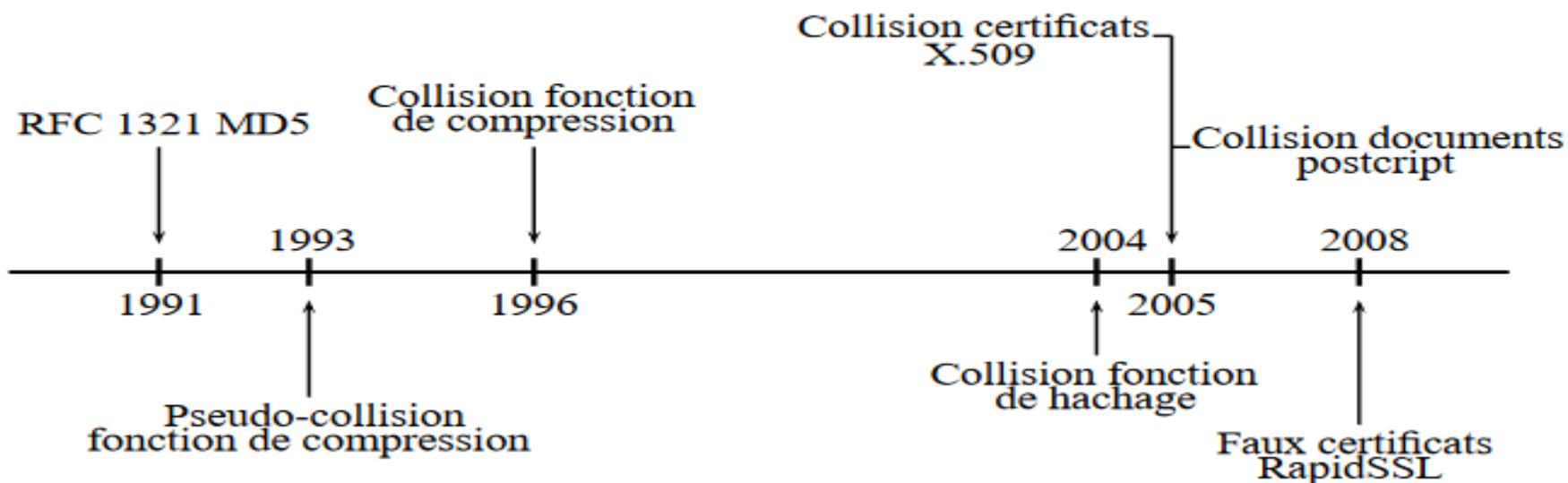
$x_1 =$	d131dd02c5e6eec4	693d9a0698aff95c	2fcab58712467eab	4004583eb8fb7f89
	55ad340609f4b302	83e488832571415a	085125e8f7cdc99f	d91dbdf280373c5b
	d8823e3156348f5b	ae6dacd436c919c6	dd53e2b487da03fd	02396306d248cda0
	e99f33420f577ee8	ce54b67080a80d1e	c69821bcb6a88393	96f9652b6ff72a70
$x_2 =$	d131dd02c5e6eec4	693d9a0698aff95c	2fcab50712467eab	4004583eb8fb7f89
	55ad340609f4b302	83e4888325f1415a	085125e8f7cdc99f	d91dbd7280373c5b
	d8823e3156348f5b	ae6dacd436c919c6	dd53e23487da03fd	02396306d248cda0
	e99f33420f577ee8	ce54b67080280d1e	c69821bcb6a88393	96f965ab6ff72a70

Both inputs provide the same hash value
79054025255fb1a26e4bc422aef54eb4

Famille de fonction de hachage MD-SHA

- Sécurité de MD5. La taille d'empreinte dans MD5 est de 128 bits. La recherche de collisions par force brute (attaque de Yuval) a donc une complexité de 2^{64} calculs d'empreintes. Pourtant, plusieurs algorithmes ont été proposés récemment pour trouver des collisions avec MD5 en seulement 2^{42} opérations, voire 2^{30} . C'est pourquoi MD5 n'est plus considéré comme sûr aujourd'hui.

Vie et mort de MD5



Famille de fonction de hachage MD-SHA

- SH1(Secure Hash Algorithm) : SHA-0 a été conçu et développé par NSA en collaboration avec NIST en 1993. Cependant, des vulnérabilités de sécurité ont été découvertes, ce qui a conduit à son remplacement par SHA-1 en 1995.
- Taille du bloc : 512 bits.
- Taille de l'emprunte : 160 bits
- Chaque bloc est divisé en 16 sous bloc de 32 bits chacun. qui sont ensuite étendus en 80
- nouveaux blocs $\{W_i\}$ $0 \leq i \leq 79$. En outre, 4 constantes $\{K_i\}$ $1 \leq i \leq 3$ sont fixées. L'algorithme travaille donc sur un état de 160 bits subdivisés en 5 mots de 32 bits chacun : A, B, C, D et E (initialisés au début avec des constantes).
- Type : Merkle-Damgard.
- Nombre de round : 80.

message block
 $x^{(0)}$

512

A B C D E

round 1 (20 steps)
 $f_1, K, W[0...19]$

A B C D E

round 2 (20 steps)
 $f_2, K, W[20...39]$

A B C D E

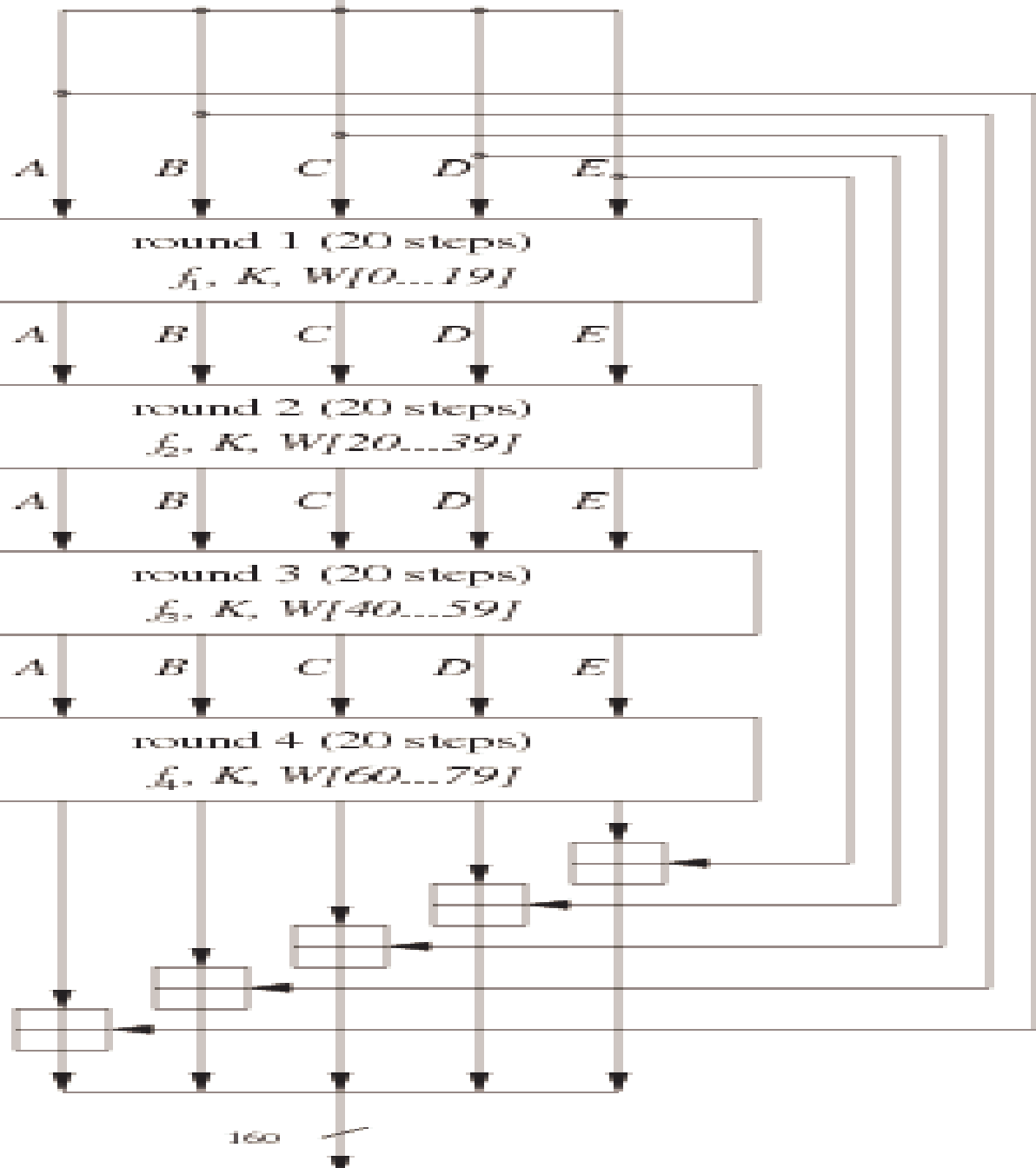
round 3 (20 steps)
 $f_3, K, W[40...59]$

A B C D E

round 4 (20 steps)
 $f_4, K, W[60...79]$

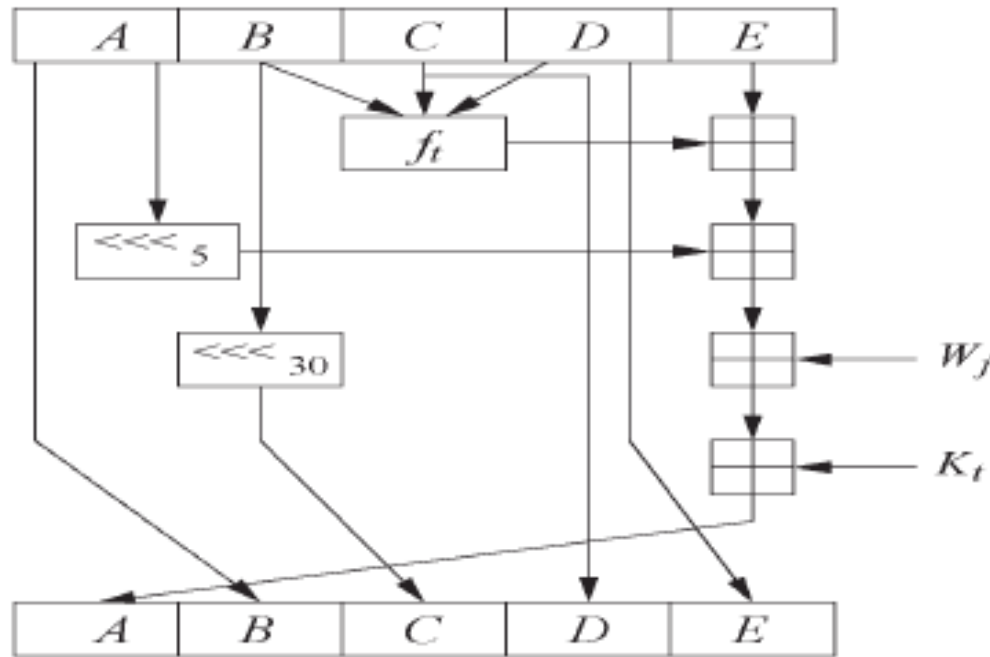
addition mod 2^{32}

160



Famille de fonction de hachage MD-SHA

- SH1(Secure Hash Algorithm): description interne de chaque round

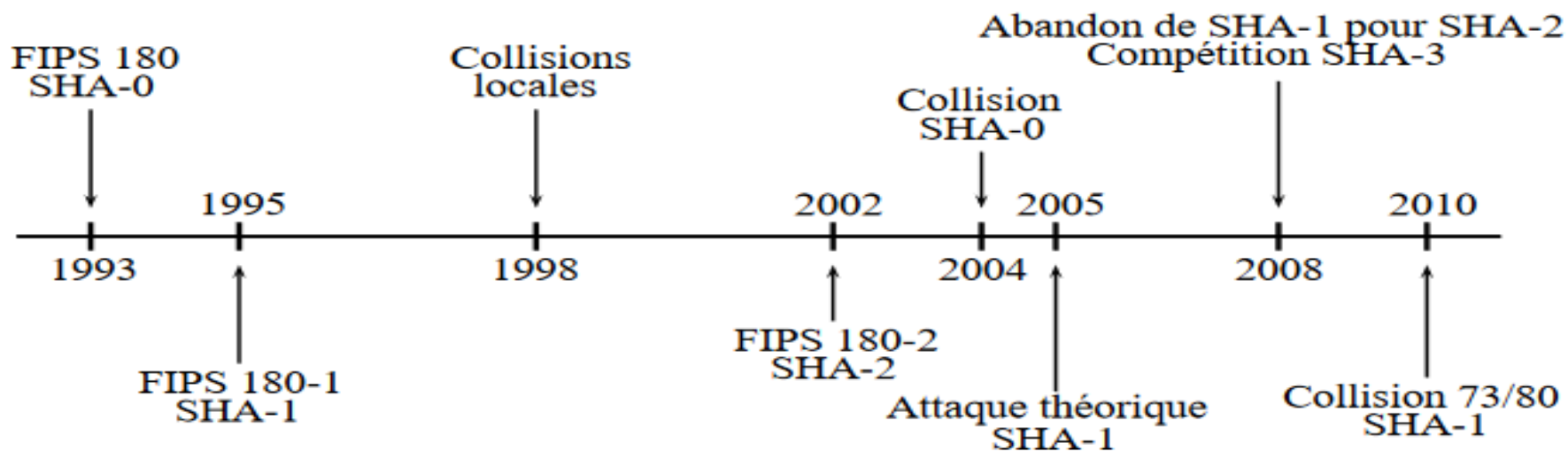


Stage t	Round j	Constant K_t	Function f_t
1	00...19	$K=5A827999$	$f(B,C,D)=(B \wedge C) \vee (\bar{B} \wedge D)$
2	20...39	$K=6ED9EBA1$	$f(B,C,D)=B \oplus C \oplus D$
3	40...59	$K=8F1BBCDC$	$f(B,C,D)=(B \oplus C) \vee (B \oplus D) \vee (C \oplus D)$
4	60...79	$K=CA62C1D6$	$f(B,C,D)=B \oplus C \oplus D$

Famille de fonction de hachage MD-SHA

- Sécurité de SHA-1. La taille d'empreinte dans SHA-1 est de 160 bits. La recherche de collisions par force brute (attaque de Yuval) a donc une complexité en 280 calculs d'empreintes. Récemment, une technique similaire à celle utilisée contre MD5 a permis d'obtenir des collisions en seulement $O(2^{63})$ calculs d'empreinte. On considère donc que SHA-1 est cassé, même si l'attaque reste difficile. De part sa popularité, cette fonction de hachage est encore largement utilisée (notamment au niveau des schémas de signature).

SHA-0 et SHA-1



Famille de fonction de hachage MD-SHA

SHA-1 Freestart Collision Example (8 October 2015)

IV1	50 6b 01 78 ff 6d 18 90 20 22 91 fd 3a de 38 71 b2 c6 65 ea
M1	9d 44 38 28 a5 ea 3d f0 86 ea a0 fa 77 83 a7 36
	33 24 48 4d af 70 2a aa a3 da b6 79 d8 a6 9e 2d
	54 38 20 ed a7 ff fb 52 d3 ff 49 3f c3 ff 55 1e
	fb ff d9 7f 55 fe ee f2 08 5a f3 12 08 86 88 a9
IV2	50 6b 01 78 ff 6d 18 91 a0 22 91 fd 3a de 38 71 b2 c6 65 ea
M2	3f 44 38 38 81 ea 3d ec a0 ea a0 ee 51 83 a7 2c
	33 24 48 5d ab 70 2a b6 6f da b6 6d d4 a6 9e 2f
	94 38 20 fd 13 ff fb 4e ef ff 49 3b 7f ff 55 04
	db ff d9 6f 71 fe ee ee e4 5a f3 06 04 86 88 ab

- Both inputs provide
f020486f071bf11053547a86f4a7153b3c950f4b
- Found by Marc Stevens, Pierre Karpman, Thomas Peyrin using
16 × 4 Nvidia GTX 970 GPUs, requires 2^{57} SHA-1 evaluations
- A freestart collision breaks the inner layer of SHA-1, does not
directly translate into a collision on the full SHA-1 because the
attacker is **not** able to choose the initial internal state

Famille de fonction de hachage MD-SHA

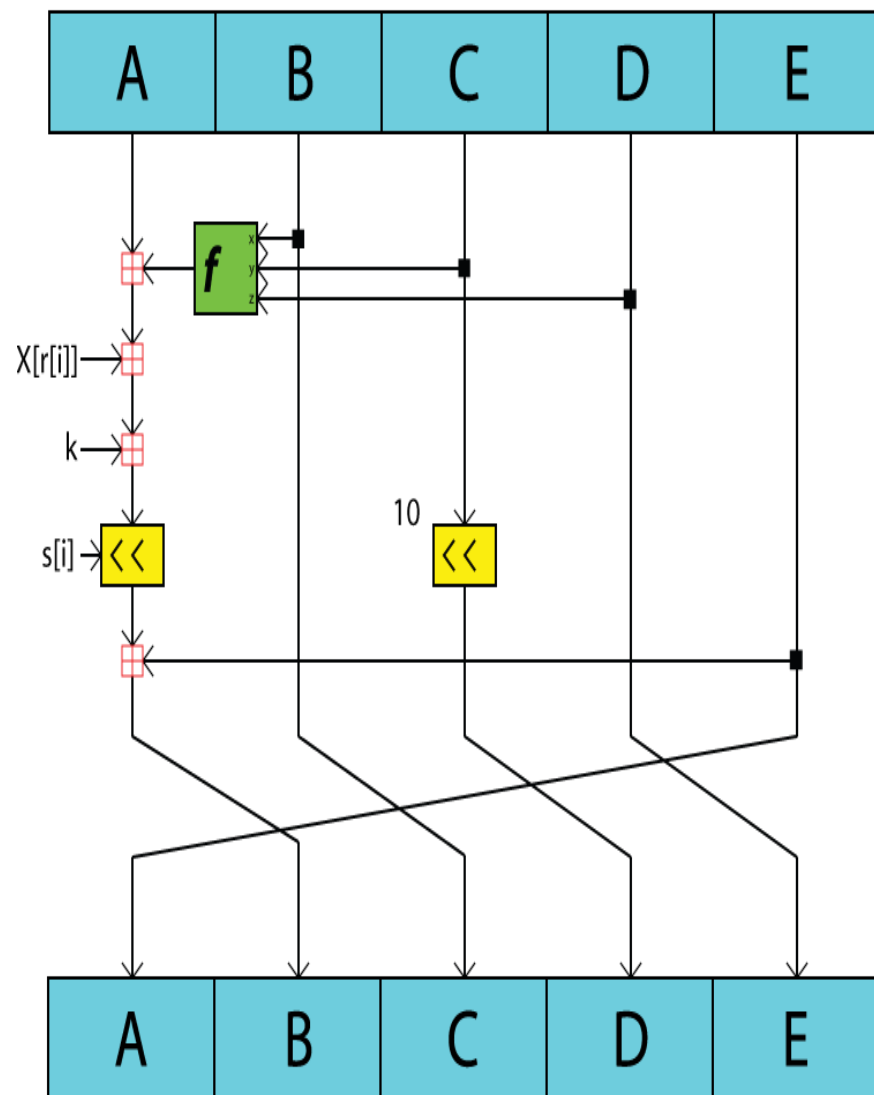
Table 1: Colliding message blocks for SHA-1.

CV_0	4e	a9	62	69	7c	87	6e	26	74	d1	07	f0	fe	c6	79	84	14	f5	bf	45
$M_1^{(1)}$			<u>7f</u>	46	dc	<u>93</u>	<u>a6</u>	b6	7e	<u>01</u>	<u>3b</u>	02	9a	<u>aa</u>	<u>1d</u>	b2	56	<u>0b</u>		
			<u>45</u>	ca	67	<u>d6</u>	<u>88</u>	c7	f8	<u>4b</u>	<u>8c</u>	4c	79	<u>1f</u>	<u>e0</u>	2b	3d	<u>f6</u>		
			<u>14</u>	f8	6d	<u>b1</u>	<u>69</u>	09	01	<u>c5</u>	<u>6b</u>	45	c1	<u>53</u>	<u>0a</u>	fe	df	<u>b7</u>		
			<u>60</u>	38	e9	<u>72</u>	<u>72</u>	2f	e7	<u>ad</u>	72	8f	0e	<u>49</u>	<u>04</u>	e0	46	<u>c2</u>		
$CV_1^{(1)}$	8d	64	<u>d6</u>	<u>17</u>	ff	ed	<u>53</u>	<u>52</u>	eb	c8	59	15	5e	c7	eb	<u>34</u>	<u>f3</u>	8a	5a	7b
$M_2^{(1)}$			<u>30</u>	57	0f	<u>e9</u>	<u>d4</u>	13	98	<u>ab</u>	<u>e1</u>	2e	f5	<u>bc</u>	<u>94</u>	2b	e3	<u>35</u>		
			<u>42</u>	a4	80	<u>2d</u>	<u>98</u>	b5	d7	<u>0f</u>	<u>2a</u>	33	2e	<u>c3</u>	<u>7f</u>	ac	35	<u>14</u>		
			<u>e7</u>	4d	dc	<u>0f</u>	<u>2c</u>	c1	a8	<u>74</u>	<u>cd</u>	0c	78	<u>30</u>	<u>5a</u>	21	56	<u>64</u>		
			<u>61</u>	30	97	<u>89</u>	<u>60</u>	6b	d0	<u>bf</u>	3f	98	cd	<u>a8</u>	<u>04</u>	46	29	<u>a1</u>		
CV_2	1e	ac	b2	5e	d5	97	0d	10	f1	73	69	63	57	71	bc	3a	17	b4	8a	c5

CV_0	4e	a9	62	69	7c	87	6e	26	74	d1	07	f0	fe	c6	79	84	14	f5	bf	45
$M_1^{(2)}$			<u>73</u>	46	dc	<u>91</u>	<u>66</u>	b6	7e	<u>11</u>	<u>8f</u>	02	9a	<u>b6</u>	<u>21</u>	b2	56	<u>0f</u>		
			<u>f9</u>	ca	67	<u>cc</u>	<u>a8</u>	c7	f8	<u>5b</u>	<u>a8</u>	4c	79	<u>03</u>	<u>0c</u>	2b	3d	<u>e2</u>		
			<u>18</u>	f8	6d	<u>b3</u>	<u>a9</u>	09	01	<u>d5</u>	<u>df</u>	45	c1	<u>4f</u>	<u>26</u>	fe	df	<u>b3</u>		
			<u>dc</u>	38	e9	<u>6a</u>	<u>c2</u>	2f	e7	<u>bd</u>	72	8f	0e	<u>45</u>	<u>bc</u>	e0	46	<u>d2</u>		
$CV_1^{(2)}$	8d	64	<u>c8</u>	<u>21</u>	ff	ed	<u>52</u>	<u>e2</u>	eb	c8	59	15	5e	c7	eb	<u>36</u>	<u>73</u>	8a	5a	7b
$M_2^{(2)}$			<u>3c</u>	57	0f	<u>eb</u>	<u>14</u>	13	98	<u>bb</u>	<u>55</u>	2e	f5	<u>a0</u>	<u>a8</u>	2b	e3	<u>31</u>		
			<u>fe</u>	a4	80	<u>37</u>	<u>b8</u>	b5	d7	<u>1f</u>	<u>0e</u>	33	2e	<u>df</u>	<u>93</u>	ac	35	<u>00</u>		
			<u>eb</u>	4d	dc	<u>0d</u>	<u>ec</u>	c1	a8	<u>64</u>	<u>79</u>	0c	78	<u>2c</u>	<u>76</u>	21	56	<u>60</u>		
			<u>dd</u>	30	97	<u>91</u>	<u>d0</u>	6b	d0	<u>af</u>	3f	98	cd	<u>a4</u>	<u>bc</u>	46	29	<u>b1</u>		
CV_2	1e	ac	b2	5e	d5	97	0d	10	f1	73	69	63	57	71	bc	3a	17	b4	8a	c5

Famille de fonction de hachage MD-SHA

- RIPMED-160: Développé par Hans Dobbertin, Antoon Bosselaers et Bart Preneel, en 1996.
- Taille du bloc : 512 bits.
- Taille de l’empreinte : 128,160,256, 320 bits
- Type : Merkle-Damgard.
- Nombre de round : 80.
- Collision résistance : 2^{80}



Famille de fonction de hachage MD-SHA

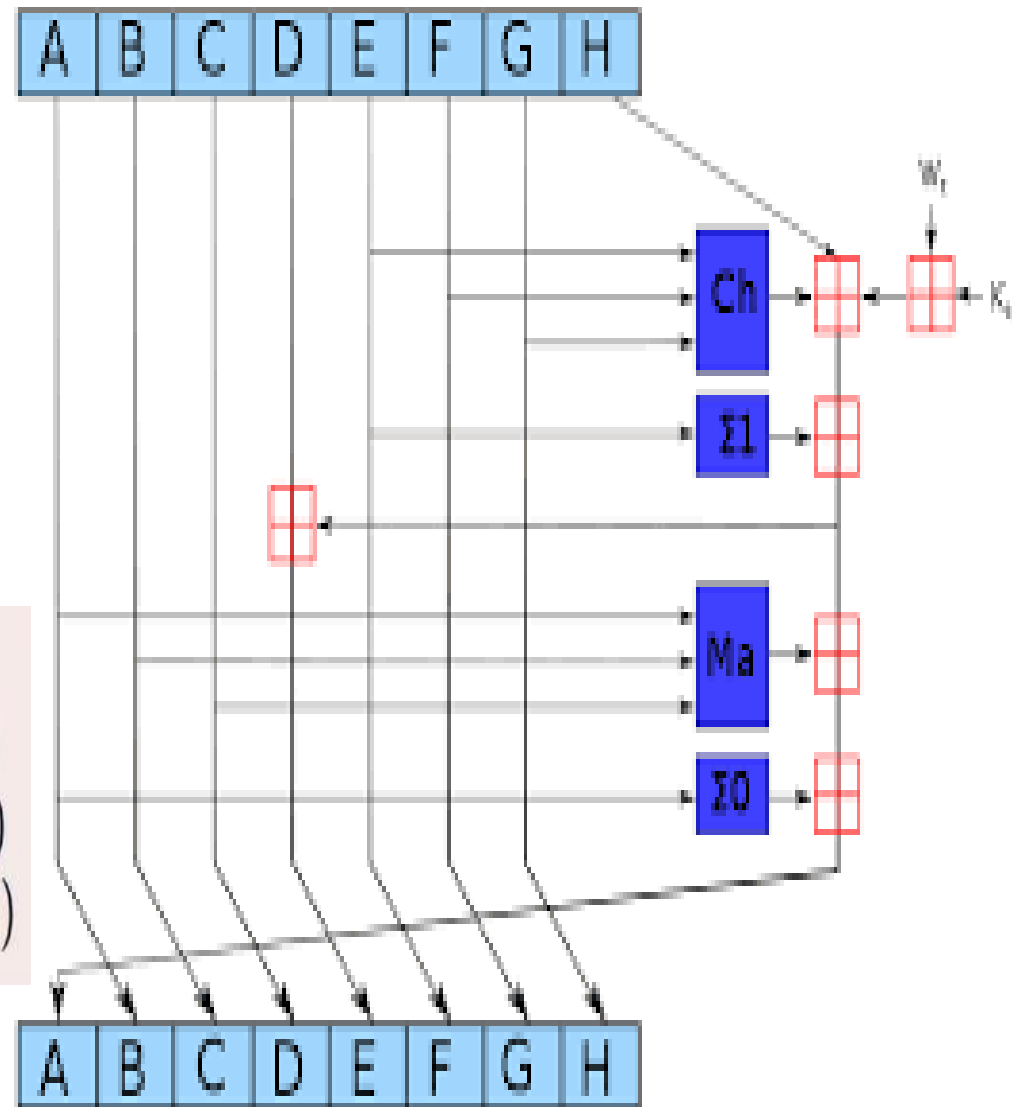
- SHA-2: Développé par NSA première publication en 2001.
- Taille du bloc : 512 ou 1024 bits.
- Taille de l'empreinte : 224, 256, 384, 512 bits
- Type : Merkle-Damgård.
- Nombre de round : 64 ou 80.

$$Ch(E, F, G) = (E \wedge F) \oplus (\neg E \wedge G)$$

$$Ma(A, B, C) = (A \wedge B) \oplus (A \wedge C) \oplus (B \wedge C)$$

$$\Sigma_0(A) = (A \gg 2) \oplus (A \gg 13) \oplus (A \gg 22)$$

$$\Sigma_1(E) = (E \gg 6) \oplus (E \gg 11) \oplus (E \gg 25)$$



Famille de fonction de hachage MD-SHA

- " Attacks against Up to 46 Rounds of SHA-2" publié par Wang et Yu en 2009. Cette attaque concerne les fonctions de hachage SHA-224, SHA-256, SHA-384 et SHA-512 et consiste en une technique qui permet de trouver des pseudo-collisions pour les fonctions jusqu'à 46 rounds (ou itérations) de ces fonctions de hachage.
- Cependant, il convient de noter que cette attaque n'affecte pas la sécurité des fonctions de hachage SHA-2 dans leur utilisation normale. Les attaques pseudo-collision sont considérées comme des attaques théoriques qui ont peu d'applications pratiques. De plus, la sécurité des fonctions SHA-2 est encore considérée comme très robuste dans la plupart des cas d'utilisation courants.
- Pre-image résistance une attaque en 2011 casse la pre-image résistance pour 57 sorties sur les 80 rounds de SHA-512 et 52 sorties sur les 64 rounds de SHA-256

Famille de fonction de hachage MD-SHA

- Compétition lancée par le NIST. Détermination du vainqueur en 2012. 5 Finalistes au bout de 2 tours (et 2 ans d'attaques intenses de la part de la communauté scientifique) :
 - ❖ Blake.
 - ❖ JH.
 - ❖ Keccak.
 - ❖ Skein.
 - ❖ Grstl.
- Keccak devenu SHA3 a été décrite par Bertoni, Daemen (coinventeur de l'AES !), Peeters et Van Assche.
- Keccak est une **fonction éponge**.

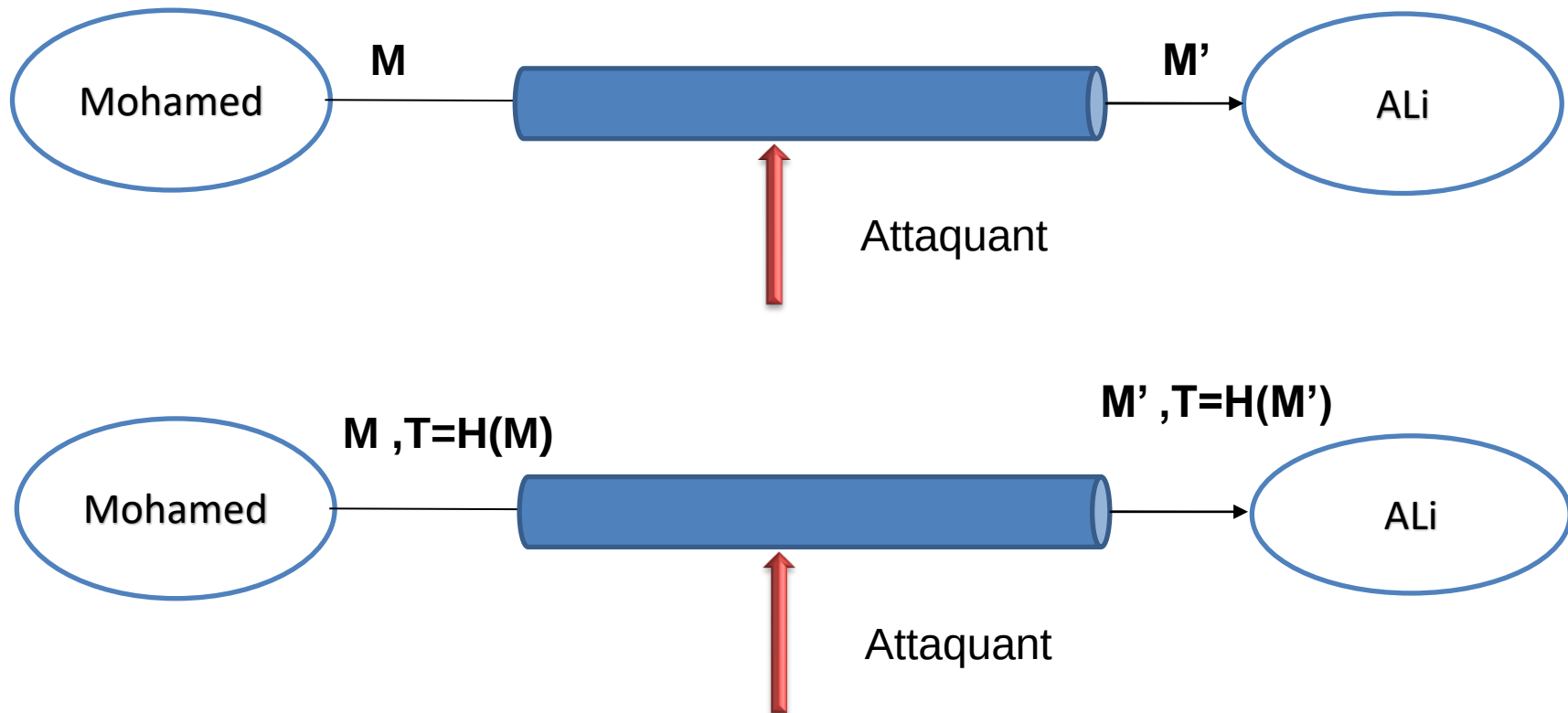
Famille de fonction de hachage MD-SHA

- Une fonction éponge c'est une amélioration d'une fonction de compression.
- **Originalité** : prendre des tailles aléatoires et sortir des tailles fixes.
- Generalise a la fois les **fonctions de hachage** et le **chirement par flot**.
- Permet de construire des **preuves de sécurité**.
- Fonctionnement : A partir d'une chaîne de r blocs, et d'un état initial = état interne
 - **Absorption** : XOR du bloc 1 avec les r premiers bits de l'état, puis fonction f sur tout l'état, puis XOR du bloc 2 etc.
 - **Essorage** : on construit pas a pas les bits de la sortie. La taille de la chaîne peut être choisie.
- Ce qu'il faut définir : qui est cette fonction f de l'absorption.

Famille de fonction de hachage MD-SHA

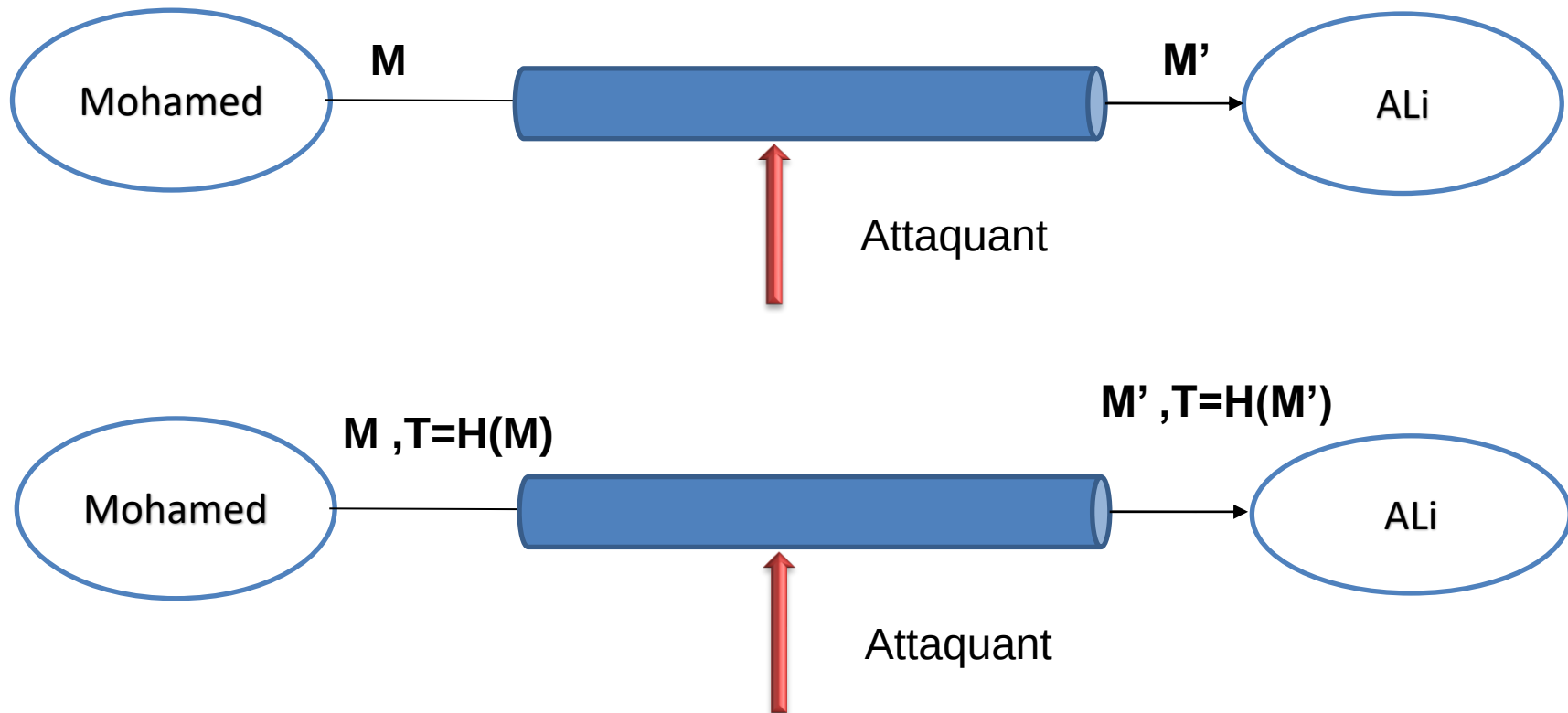
- Fonction éponge.
- Empreinte de **taille variable** : l'utilisateur choisit. SHA3-356, SHA3-512...
- **Efficace** et rapide (plus que SHA2).
- **Effet avalanche** : changer un seul bit dans le message en modifie en moyenne la moitié dans l'empreinte.
- **Sécurité** qui dépend de la version. Par exemple pour SHA3-256 :
 - Résistance aux attaques par collision : 2^{128}
 - Résistance aux attaques par pré-image et seconde pré-image : 2^{256}
 - Et les attaques quantiques ? Collisions en 2^{85} .
 - Pour résister au quantique → SHA3-384 minimum.

Message Authentication Codes (MAC)



- Le but est de s'assurer que le message reçu par Ali vient de Mohamed 'authentication' et que le message n'a pas été changé lors de l'envoi à travers un canal non sécurisé.

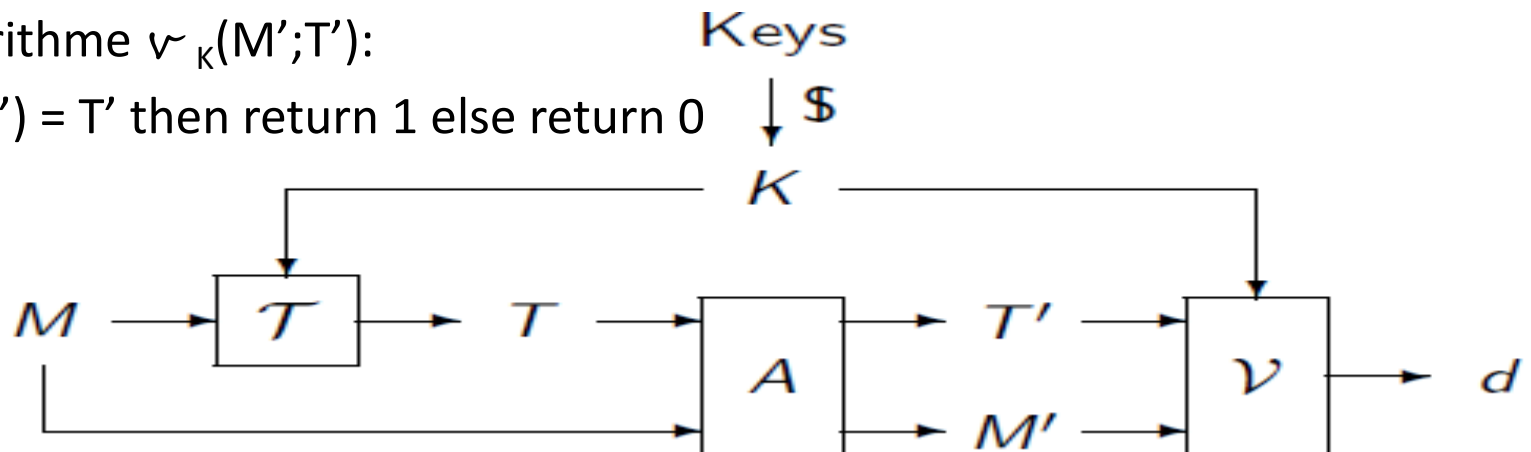
Message Authentication Codes (MAC)



- Exemple mohamed dit à ali qu'il doit donner mohstapha 1000 da, l'attaquant pourra changer juste la somme d'argent en la rendant 10000 da.

Message Authentication Codes (MAC)

- Un code d'authentification de message (en abrégé CAM ou MAC Message Authentication Code) est un dispositif permettant de garantir l'intégrité des données transmises à travers un canal non sécurisé entre deux entités, c'est à dire de garantir que les données n'ont pas été modifiées pendant leur acheminement.
- Mac est similaire à la signature numérique sauf qu'elle utilise le chiffrement symétrique une seule clé est partagée entre émetteur et destinataire.
- *soit* un message authentication code $\mathcal{T}: K \times T \rightarrow R$ et soit A un attaquant
- ✓ T réfère au Tag généré par $\mathcal{T}$.
- Algorithme $\mathcal{V}_K(M'; T')$:
If $\mathcal{T}_K(M') = T'$ then return 1 else return 0

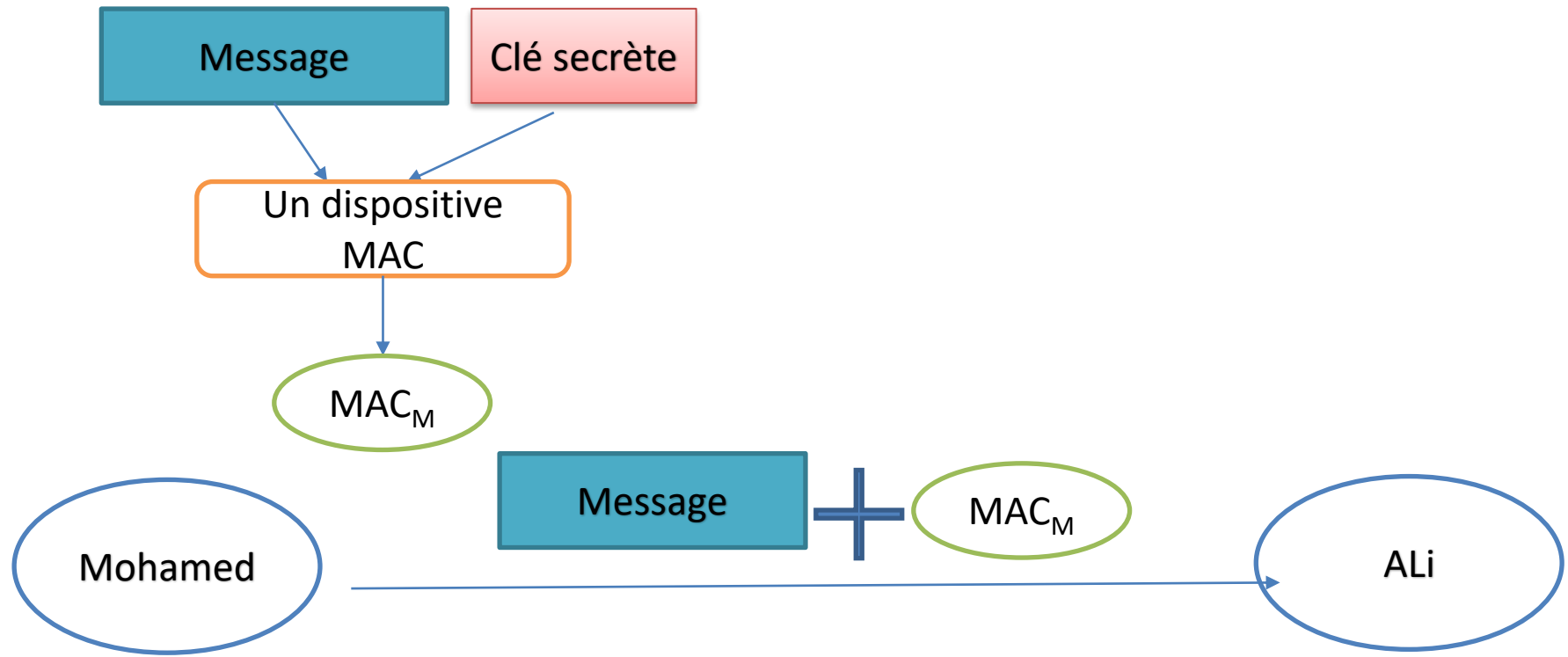


Message Authentication Codes (MAC)

- Propriétés du MAC:

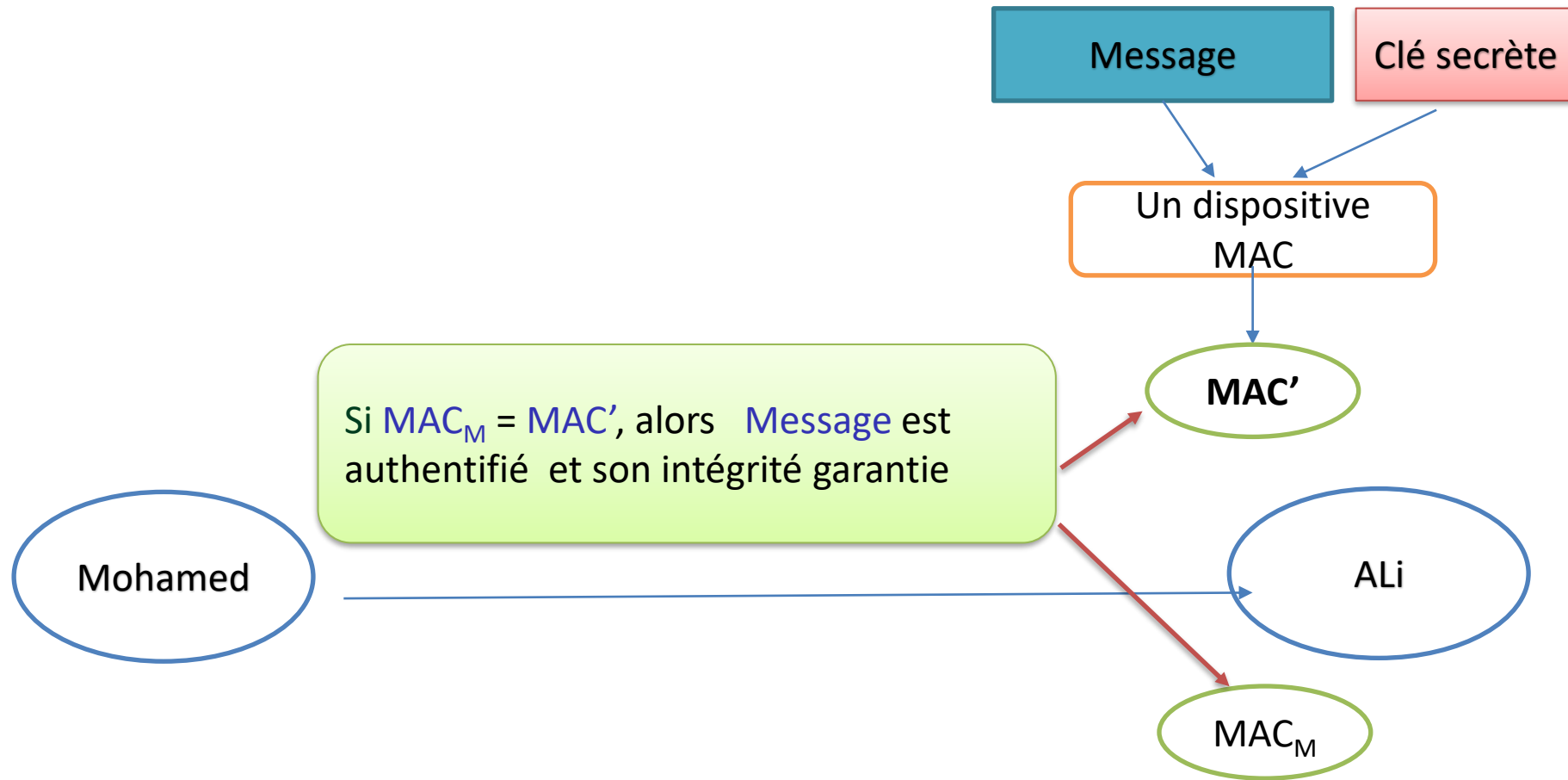
1. **checksum cryptographique**: Un MAC génère une étiquette d'authentification cryptographiquement sûre pour un message donné.
2. **symétrique**: Les MAC sont basés sur des clés symétriques secrètes. Les parties signataires et vérificatrices doivent partager une clé secrète.
3. **taille arbitraire des messages**: Les MAC acceptent des messages de longueur arbitraire.
4. **longueur de sortie fixe**: Les MAC génèrent des étiquettes d'authentification de taille fixe.
5. **intégrité des messages**: Les MAC assurent l'intégrité des messages : Toute manipulation d'un message pendant le transit sera détectée par le destinataire.
6. **Authentification du message**: Le destinataire est assuré de l'origine du message.
7. **non-répudiation**: Les MAC étant basés sur des principes symétriques, ils n'assurent pas la non-répudiation.

Message Authentication Codes (MAC)



- Schéma de fonctionnement du MAC

Message Authentication Codes (MAC)



- Schéma de fonctionnement du MAC

Message Authentication Codes (MAC)

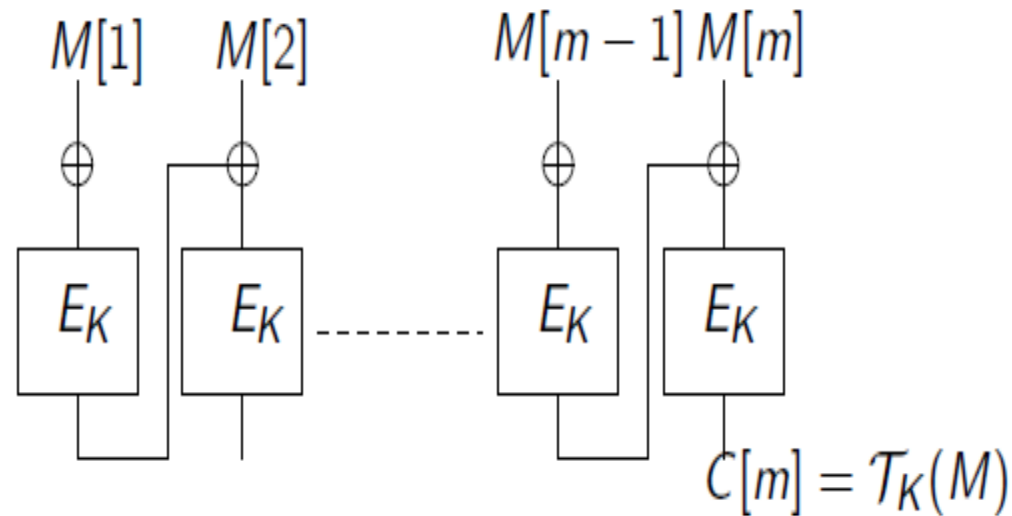
- Les MAC peuvent être construits à partir de chiffrement par blocs. La construction la plus importante est le CBC-MAC. Soit E la fonction de chiffrement d'un chiffrement par blocs, tel que DES ou AES. Ensuite, avec k une clé secrète, la valeur MAC pour un message m est le dernier bloc de texte chiffré lors du chiffrement de m , avec E dans le mode de chiffrement par blocs (CBC-MAC).
- Let $E : \{0; 1\}^k * B \rightarrow B$ est un bloc du ciphertext, tel que $B = \{0; 1\}^n$. le message $M \in B^*$ est une sequence de block de n -bit, $M = M[1] \dots M[m]$.
- The basic CBC MAC $\mathcal{T} : \{0; 1\}^k * B \rightarrow B$ depend de algorithme suivant

Alg $\mathcal{T}_k(M)$

$C[0] \leftarrow 0^n$

for $i = 1; \dots; m$ do $C[i] \leftarrow E_k(C[i-1], M[i])$

return $C[m]$



Message Authentication Codes (MAC)

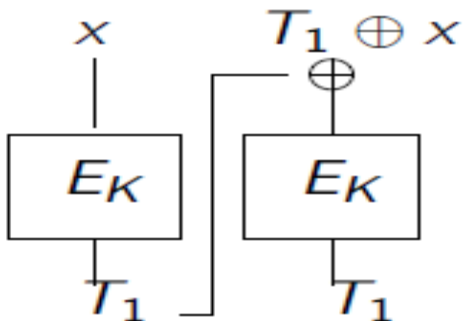
- L'attaque de splice (splicing) sur le chiffrement CBC-MAC (Cipher Block Chaining Message Authentication Code) est une attaque qui consiste à modifier un message en injectant un autre message sans avoir accès à la clé de chiffrement. Cette attaque est possible car le chiffrement CBC-MAC utilise la même clé de chiffrement pour le chiffrement et la génération du code d'authentification du message. Ainsi, un attaquant peut modifier un bloc de message en calculant un nouveau bloc de code d'authentification à partir de ce bloc modifié et des blocs de message qui le précèdent. Cette attaque peut être utilisée pour modifier le message de manière malveillante ou pour contourner la vérification d'intégrité du message par le destinataire.

Alg $T_K(M)$ s

$C[0] \leftarrow 0^n$

for $i = 1; \dots; m$ do $C[i] \leftarrow E_K(C[i-1], M[i])$

return $C[m]$



adversary A

Let $x \in \{0; 1\}^n$

$T_1 \leftarrow \text{Tag}(x)$

$M \leftarrow x || T_1 \oplus x$

Return $M \cdot T_1$

$$\begin{aligned}
 T_K(M) &= E_K(E_K(x) \oplus T_1 \oplus x) \\
 &= E_K(T_1 \oplus T_1 \oplus x) \\
 &= E_K(x) \\
 &= T_1
 \end{aligned}$$

Message Authentication Codes (MAC)

- Interdiction d'utiliser la même clé pour ciphérer par bloc un message et pour en extraire un CBC-MAC. Sinon attaquant peut créer des faux MAC par adaptation du dernier bloc.
- Comme souvent il faut être vigilant avec l'IV : ici, il doit vraiment être initialisé à 0. Sinon m_0 n'est pas authentifié.
- Attention, si la taille du message est indiquée nulle part, attaquant peut concaténer deux messages dont il connaît les MAC, et produire un 3e MAC valable.

Message Authentication Codes (MAC)

- Supposons qu'un attaquant veuille modifier le message "Transfer 1000 dollars to account A" en "Transfer 10000 dollars to account B", protégé par un MAC en utilisant CBC-MAC. Voici comment l'attaque pourrait se dérouler :
 1. L'attaquant intercepte le message original "Transfer 1000 dollars to account A" et le MAC correspondant.
 2. L'attaquant divise le message original en deux blocs de 8 octets chacun (supposons que la taille de bloc pour CBC-MAC soit de 8 octets), soit : "Transfer " et "1000 dol".
 3. L'attaquant calcule le CBC-MAC pour le deuxième bloc "1000 dol" en utilisant la clé et le vecteur d'initialisation du message original qu'il a intercepté.
 4. L'attaquant remplace le deuxième bloc du message original par le bloc qu'il vient de calculer, obtenant ainsi le message modifié : "Transfer 10000 dollars to account A" et le MAC correspondant.
 5. L'attaquant envoie le message modifié et le MAC correspondant au destinataire.
 6. Le destinataire vérifie le MAC pour le message modifié et constate que le MAC est valide, car il est calculé en utilisant les deux premiers blocs du message original et le vecteur d'initialisation correspondant. Le destinataire ne détecte pas la modification apportée au message.
- Ainsi, l'attaquant a réussi à modifier le message sans être détecté en utilisant une attaque de "splicing" sur le CBC-MAC.

Message Authentication Codes (MAC)

- Interdiction d'utiliser la même clé pour ciphérer par bloc un message et pour en extraire un CBC-MAC. Sinon attaquant peut créer des faux MAC par adaptation du dernier bloc.
- Comme souvent il faut être vigilant avec l'IV : ici, il doit vraiment être initialisé à 0. Sinon m_0 n'est pas authentifié.
- Attention, si la taille du message est indiquée nulle part, attaquant peut concaténer deux messages dont il connaît les MAC, et produire un 3e MAC valable.

Message Authentication Codes (MAC)

HMAC : Keyed-Hashing for Message Authentication

- HMAC est le code d'authentification de message le plus utilisé à l'heure actuelle (normalisé dans le [RFC-2104](#)).
- N'importe quelle fonction (itérative) de hachage peut-être utilisée avec HMAC. On parle alors de HMAC-MD5, HMAC-SHA-1, HMAC-SHA-224,...
- Soit h une fonction de hachage opérant sur des blocs de s octets (par exemple, $s = 64$ pour $h = \text{MD5}$ ou $h = \text{SHA-256}$) et soient m , k respectivement le message à authentifier et la clé secrète, on a :

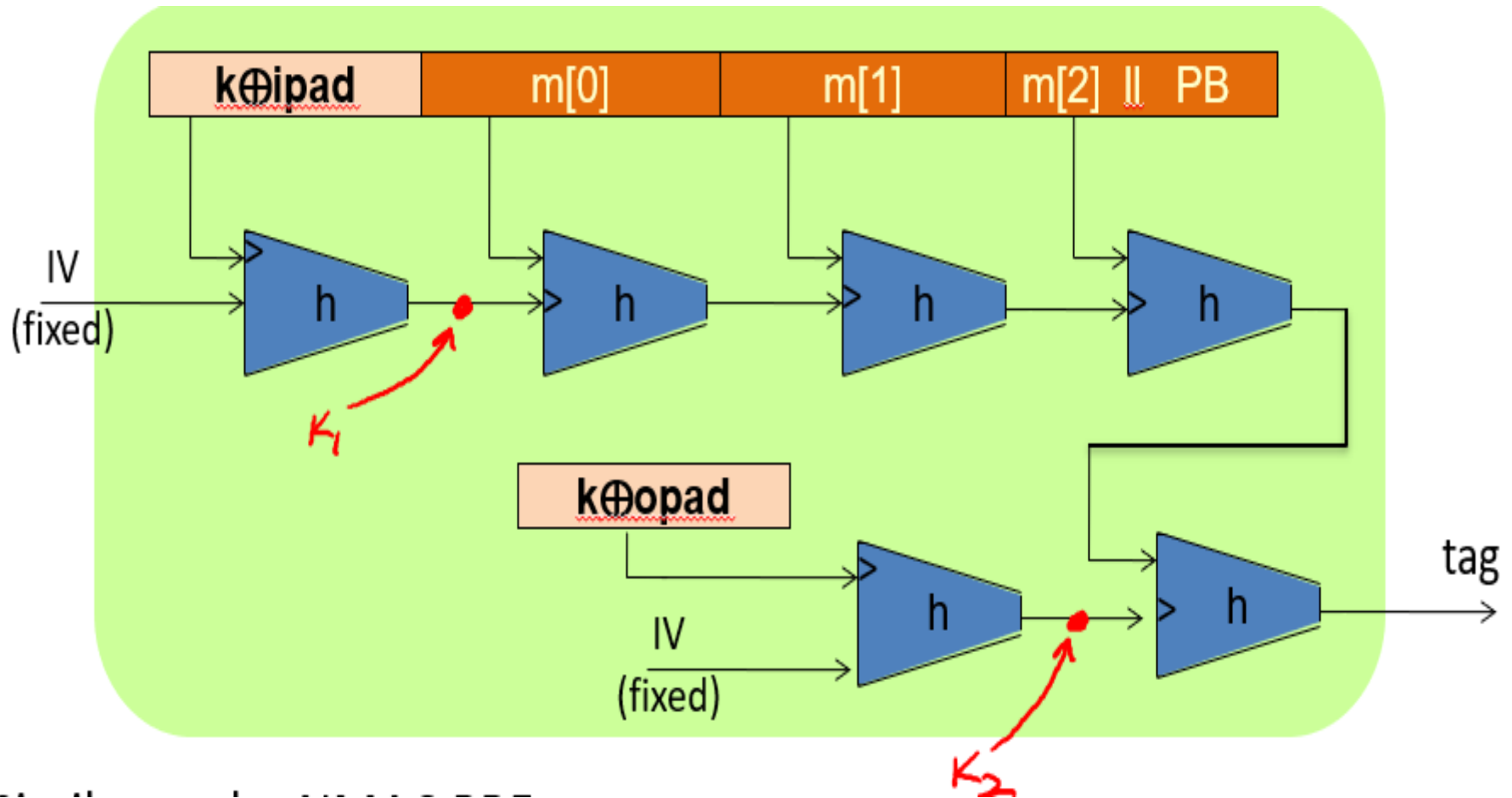
$$\text{HMAC}_h(m, k) = h((k' \oplus \text{opad}) \parallel h((k' \oplus \text{ipad}) \parallel m)),$$

avec :

- $k' = \overbrace{k \parallel 0 \cdots 0}^{s \text{ octets}}$ si $\text{taille}(k) \leq s$. Si $\text{taille}(k) > s$, $k' = h(k)$.
- $\text{opad} = 0x \underbrace{5c \cdots 5c}_{s \text{ octets}}$, $\text{ipad} = 0x \underbrace{36 \cdots 36}_{s \text{ octets}}$

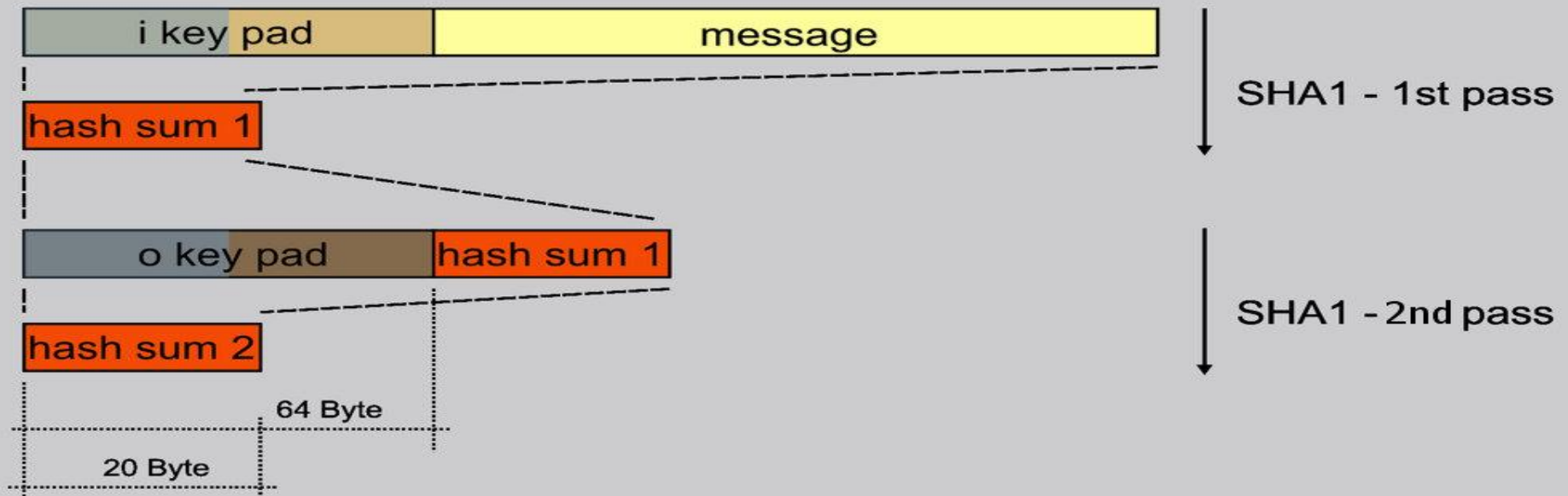
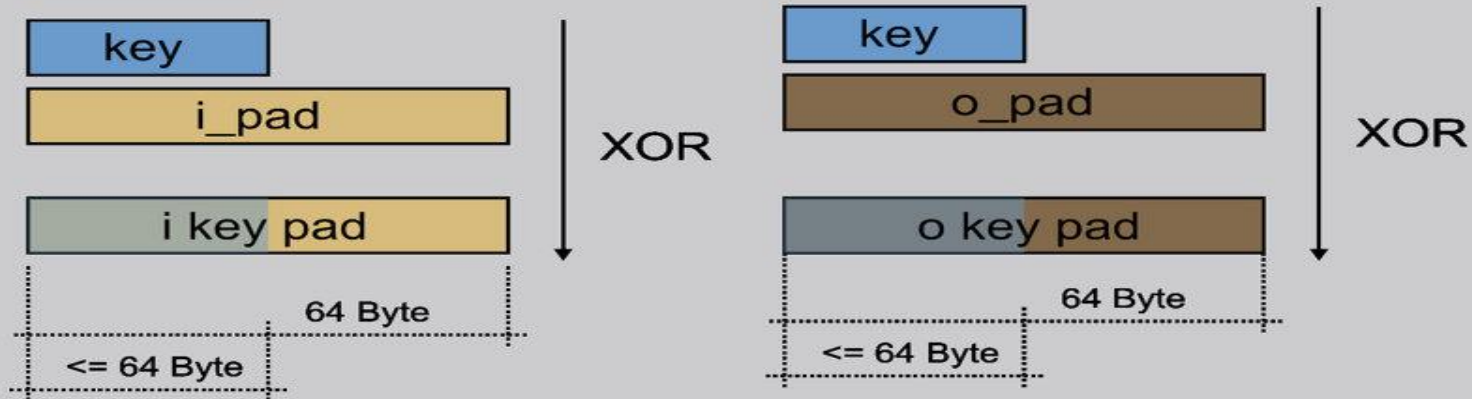
Message Authentication Codes (MAC)

HMAC : Keyed-Hashing for Message Authentication



Message Authentication Codes (MAC)

HMAC : Keyed-Hashing for Message Authentication



Message Authentication Codes (MAC)

